AUDITORÍA TÉCNICA ESPECIALIZADA

# INFORME DE AUDITORÍA INTEGRAL DE ARQUITECTURA, CÓDIGO FUENTE, SEGURIDAD Y BASES DE DATOS

Inspección quirúrgica al 100% del sistema POS multi-empresa construido en Laravel 11/13, Blade, Bootstrap 5 y motor relacional con control fiscal.

| | |
|---|---|
| **Plataforma:** | Sistema POS Multi-Empresa con Facturación y Control Fiscal |
| **Framework / Lenguaje:** | Laravel 11 / PHP 8.4 - 8.5 • Blade • Bootstrap 5 • jQuery |
| **Motor de Datos:** | Base de Datos Relacional (MySQL / MariaDB InnoDB con Foreign Keys) |
| **Seguridad / Permisos:** | Spatie Laravel-Permission • RBAC Multi-Inquilino |
| **Tipo de Auditoría:** | Code Review Exhaustivo, Concurrencia, Transacciones y UI |
| **Alcance Evaluado:** | Facturación, Secuencias, Kardex, Cajas, CXC, CXP, Roles y Vistas |
| **Dictamen General:** | **APROBADO CON RECOMENDACIONES (Madurez Alta)** |

Documento técnico generado automáticamente tras el análisis estático y dinámico del repositorio.

# 1. RADIOGRAFÍA TECNOLÓGICA Y ESTRUCTURAL

El sistema bajo auditoría es una solución comercial y de punto de venta (POS) multi-inquilino segregada lógicamente mediante el discriminador `empresa_id`. La arquitectura sigue los principios de **Controladores Delgados (Thin Controllers)**, **Servicios de Dominio** dedicados (ubicados en `app/Service/**`), **FormRequests** con validaciones en servidor y **Modelos Eloquent** con integridad referencial garantizada por llaves foráneas a nivel de motor relacional.

## 1.1. Matriz Real de Modelos Eloquent y Tablas de Base de Datos

| MODELO ELOQUENT | TABLA REAL BD | RELACIONES CARDINALES | INTEGRIDAD REFERENCIAL / ÍNDICES |
|---|---|---|---|
| `Venta` | `ventas` | BelongsTo(Empresa, Cliente, Almacen, Caja, CajaTurno, User) HasMany(VentaDetalle, VentaPago, Devolucion) | FK restrict en cliente, almacén, caja, usuario. **Unique:** `(empresa_id, codigo)`, `(empresa_id, numero_control)` |
| `VentaDetalle` | `venta_detalles` | BelongsTo(Venta, Producto, Moto, Servicio, Almacen) | FK cascade en venta_id; restrict en producto_id y almacen_id. Index: `(venta_id, producto_id)` |
| `Secuencia` | `secuencias` | BelongsTo(Empresa) | Control de cerrojo exclusivo pesimista ( `lockForUpdate` ). **Unique:** `(empresa_id, tipo)` |
| `ProductoStockAlmacen` | `producto_stock_almacenes` | BelongsTo(Producto, Almacen) | Control de existencias físicas por almacén. **Unique:** `(producto_id, almacen_id)` |
| `Kardex` | `kardex` | BelongsTo(Empresa, Almacen, Producto, User) | Libro mayor de movimientos inmutables. Index: `(empresa_id, producto_id, almacen_id)` |
| `DevolucionVenta` | `devoluciones_venta` | BelongsTo(Empresa, Venta, CajaTurno, User) HasMany(DevolucionVentaDetalle) | FK restrict en venta_id, turno_id. **Unique:** `(empresa_id, codigo)` |
| `CuentaPorCobrar` | `cuentas_por_cobrar` | BelongsTo(Empresa, Cliente, Venta) HasMany(CuentaPorCobrarAbono) | FK restrict en cliente_id, venta_id. **Unique:** `(empresa_id, venta_id)` |
| `CuentaPorCobrarAbono` | `cuentas_por_cobrar_abonos` | BelongsTo(CuentaPorCobrar, User, MetodoPago, CajaTurno) | FK restrict en user_id, metodo_pago_id, caja_turno_id. Entrada directa al arqueo de caja. |

| MODELO ELOQUENT | TABLA REAL BD | RELACIONES CARDINALES | INTEGRIDAD REFERENCIAL / ÍNDICES |
|---|---|---|---|
| `CuentaPorPagar` | `cuentas_por_pagar` | BelongsTo(Empresa, Proveedor, Recepcion) HasMany(CuentaPorPagarAbono) | FK restrict en proveedor_id, recepcion_id. Index: `(empresa_id, proveedor_id, estado)` |
| `CuentaPorPagarAbono` | `cuentas_por_pagar_abonos` | BelongsTo(CuentaPorPagar, User, MetodoPago) | FK restrict en user_id, metodo_pago_id. Index: `(cuenta_por_pagar_id, fecha_abono)` |
| `CajaTurno` | `caja_turnos` | BelongsTo(Empresa, Caja, User) HasMany(Venta, CajaMovimiento) | FK restrict en caja_id, user_id (auditoría protegida). Index: `(empresa_id, estado)` |
| `CajaMovimiento` | `caja_movimientos` | BelongsTo(Empresa, CajaTurno, User, MetodoPago) | Libro de efectivo del turno (ingreso/egreso/vueltos/reembolsos). Index: `(caja_turno_id, tipo)` |

**Integridad Referencial Auditada:** Se verificó la migración `2026_11_07_201725`, la cual configuró las claves foráneas de auditoría ( `ventas.user_id` , `ventas.anulada_por_id` y `caja_turnos.user_id` ) con restricción estricta ( `restrictOnDelete` ). Ningún usuario que haya facturado, cobrado o cerrado turno puede ser eliminado físicamente de la base de datos, blindando la trazabilidad contable.

# 2. DIAGRAMAS DE FLUJO Y CONTROL DE CONCURRENCIA

## 2.1. Proceso Transaccional de Facturación y Prevención de Condiciones de Carrera

El flujo de emisión implementa cerrojos pesimistas (`lockForUpdate`) y transacciones ACID en el método `VentaClass::procesarVenta()`:

1. **Recepción de Solicitud y Validación FormRequest**

   `PosController::guardar()` recibe el payload y lo valida con `GuardarVentaRequest`. Se valida sintaxis, cliente activo, almacén de la empresa y turno de caja operativo abierto.

2. **Apertura de Transacción ACID (DB::transaction)**

   Se inicia la transacción de base de datos. Cualquier falla o excepción posterior gatilla un `ROLLBACK` automático integral, impidiendo inconsistencias parciales.

3. **Generación de Correlativo Interno bajo Cerrojo Pesimista**

   `Secuencia::siguiente($empresaId, 'venta')` ejecuta `SELECT * FROM secuencias WHERE empresa_id = ? AND tipo = 'venta' FOR UPDATE`. Se incrementa `ultimo_valor` y se formatea como `VEN-00045`. Si dos cajeros facturan al mismo tiempo, el segundo espera la liberación del cerrojo.

4. **Generación de Número de Control Fiscal bajo Cerrojo Pesimista**

   `Secuencia::siguiente($empresaId, 'numero_control')` bloquea la fila del control fiscal. Formatea el número a 8 dígitos: `00-00000045`. Los índices únicos en `ventas` impiden cualquier duplicado físico.

5. **Bloqueo y Descuento de Stock en Almacén**

   Por cada producto, `KardexClass::registrarMovimiento()` bloquea la fila en `producto_stock_almacenes` con `lockForUpdate()`. Si la existencia es menor a la requerida, aborta con excepción. Si es suficiente, descuenta el stock y escribe el asiento inmutable en `kardex` (tipo `salida_venta`). Si maneja seriales o es moto, actualiza a `vendido`.

6. **Persistencia de Venta, Pagos y Asientos Derivados**

   Inserta la cabecera en `ventas` y renglones en `venta_detalles` y `venta_pagos`. Si el cliente pagó en exceso, crea un egreso por vuelto en `caja_movimientos`. Si quedó saldo pendiente, crea la cuenta en `cuentas_por_cobrar`. Si provenía de una preventa en espera, elimina la fila de `ventas_en_espera`.

7. **COMMIT de Base de Datos y Retorno**

   El motor de BD ejecuta el `COMMIT` liberando los cerrojos. Retorna HTTP 200 con la venta lista para impresión en ticket térmico o formato carta.

## 2.2. Proceso de Anulación de Facturas y Reversión de Stock

Ejecutado por `FacturaController::anular()` y `VentaClass::anularVenta()` bajo `DB::transaction()`:

1. **Validación de Permisos y Turno Activo**

   `AnularFacturaRequest` exige el permiso `ventas.anular`, un motivo obligatorio ≥ 5 caracteres y la existencia de un turno de caja abierto para registrar el desembolso de reembolso.

2. **Bloqueo Pesimista de la Venta**

   Se ejecuta `Venta::where('id', $id)->lockForUpdate()->firstOrFail()`. Se comprueba que el estado sea `completada` o `devuelta_parcial`. Si ya está anulada o devuelta total, se rechaza.

3. **Reintegro de Existencias Físicas en Kardex**

   Por cada renglón en `venta_detalles`, se calcula la cantidad pendiente: `cantidad - devuelto_previo`. Si es mayor a 0, `KardexClass` reintegra el stock al almacén con tipo de movimiento `anulacion_venta`. Los seriales únicos y motos regresan a estado `disponible`.

4. **Anulación de Cuentas por Cobrar (CXC)**

   Si la venta tenía crédito asociado en `cuentas_por_cobrar`, se bloquea y se actualiza a `estado = 'anulada'` y `saldo_pendiente = 0`.

5. **Reembolso Financiero en Turno Activo**

   Se calcula el dinero neto efectivamente cobrado (pagos de la venta + abonos de CXC − reembolsos previos − vuelto entregado). Se registra un movimiento de `egreso` en `caja_movimientos` asignado al **turno abierto actual** por cada método y moneda.

6. **Marcado de Estado y Auditoría (Sin Destrucción de Datos)**

   La venta se actualiza con `estado = 'anulada'`, `anulada_por_id = $userId`, `anulada_at = now()`, `motivo_anulacion = $motivo`, `saldo_pendiente = 0` y `comision_monto = 0`.
   **Preservación de Consecutivos:** Ni el `codigo` ni el `numero_control` se borran ni se liberan. No hay huecos fiscales.

# 3. ANÁLISIS QUIRÚRGICO POR MÓDULO

## 3.1. Módulo de Facturación y Control Fiscal

**Archivos Responsables:** `FacturaController.php`, `PosController.php`, `VentaClass.php`, `Secuencia.php`, `GuardarVentaRequest.php`, `AnularFacturaRequest.php`.
**Tablas Afectadas:** `ventas`, `venta_detalles`, `venta_pagos`, `ventas_en_espera`, `secuencias`.

- **Cálculo de Precios e Impuestos:** Valida precios de lista en servidor. Si el precio difiere o tiene descuento, exige `ventas.descuentos`. Calcula base imponible desglosando el 16% de IVA sobre el neto. Prohíbe que el precio de venta neto quede por debajo del costo unitario registrado (`costo_unitario_usd`).
- **Control de Concurrencia:** Implementado en `Secuencia::siguiente()`. La tabla `secuencias` actúa como monitor mutex. Si dos cajeros facturan simultáneamente en la misma empresa, la base de datos serializa los accesos sin generar saltos de correlativo.
- **Inmutabilidad Fiscal:** Ante una anulación, la factura se conserva formalmente en la base de datos con sello de anulación y motivo. El número de control permanece asignado a ese documento y nunca se recicla.

## 3.2. Módulo de Almacenes e Inventarios

**Archivos Responsables:** `KardexClass.php`, `KardexController.php`, `RecepcionClass.php`.
**Tablas Afectadas:** `producto_stock_almacenes`, `kardex`, `producto_seriales`, `motos`, `recepciones`, `recepcion_detalles`.

- **Mecanismo de Descuento:** Orquestado en backend dentro de `KardexClass::registrarMovimiento()`. No se emplean triggers de base de datos ni Observers de Eloquent, garantizando control 100% transaccional en PHP.
- **Operación Matemática:** En `producto_stock_almacenes`, si el movimiento es entrada (`entrada_recepcion`, `traslado_entrada`, `ajuste_positivo`, `anulacion_venta`), ejecuta `$stockNuevo = $stockAnterior + $cantidad`. Si es salida (`salida_venta`, `traslado_salida`, `ajuste_negativo`), ejecuta `$stockNuevo = $stockAnterior - $cantidad`, arrojando excepción de bloqueo si `$stockAnterior $cantidad`.
- **Kardex Inmutable:** Cada operación crea un asiento histórico permanente con `stock_anterior`, `stock_nuevo`, costos en USD/Bs y motivo.

## 3.3. Módulo de Cuentas por Pagar (CXP) y Finanzas

**Archivos Responsables:** `CuentaPorPagarController.php`, `CuentaPorPagarClass.php`, `RecepcionClass.php`.
**Tablas Afectadas:** `cuentas_por_pagar`, `cuentas_por_pagar_abonos`, `recepciones`, `proveedores`.

- **Registro de la Deuda:** Las recepciones de mercancía a crédito generan automáticamente una fila en `cuentas_por_pagar` vinculada a `recepcion_id`.
- **Modalidades de Abono:** Soporta abono a factura específica y abono general de deuda por algoritmo FIFO (amortizando las facturas más antiguas primero).
- **Hallazgo Financiero:** A diferencia de CXC, los abonos a proveedores en `cuentas_por_pagar_abonos` no asientan automáticamente salidas de efectivo en `caja_movimientos` ni exigen un turno de caja abierto.

## 3.4. Seguridad, Roles y Permisología (RBAC)

**Archivos Responsables:** `RolesYPermisosSeeder.php`, `admin.php`, `UsuarioController.php`.

**Componentes de Seguridad:** Spatie Permission (53 permisos, 4 roles), Middlewares HTTP por ruta.

- **SuperAdmin:** Control global del sistema; gestión de empresas y métodos de pago.
- **Admin:** Control total de la empresa asignada; puede liquidar comisiones, anular ventas y abrir/cerrar turnos de caja.
- **Operador / Cajero:** Permisos granulares de mostrador. Cierra su turno a ciegas.
- **Vendedor:** Restringido exclusivamente a preventas ( `pos.preventa` ). El backend bloquea cualquier intento de cobro con excepción de autorización.

## 4. MATRIZ DE AUDITORÍA DE UI Y BOTONES POR MÓDULO

Inventario exhaustivo de las vistas Blade (`resources/views/Sistema/pages/**`) y sus scripts JS (`public/estilos/jsPropios/**`):

| VISTA / MÓDULO | BOTONES EXISTENTES | RUTA HTTP ASOCIADA | DIRECTIVA @CAN / PERMISO | ESTADO |
|---|---|---|---|---|
| **POS**<br>`pos.blade.php` | Cobrar / Facturar<br>Guardar en Espera<br>Cuentas en Espera<br>Atajo Corte X | POST /pos/guardar<br>POST /pos/en-espera/guardar<br>GET /pos/en-espera/lista<br>GET /cajas/turnos/{id}/reporte-x | @can('ventas.crear')<br>@can('pos.preventa')<br>@can('pos.acceso')<br>@can('cajas.arqueo') | CONFORME |
| **Facturas**<br>`facturas.blade.php` | Ver Detalle 360°<br>Reimprimir Ticket / Carta<br>**Devolver Factura**<br>**Anular Factura** | GET /facturas/{id}<br>GET /pos/imprimir-ticket/{id}<br>POST /facturas/{id}/devolver<br>POST /facturas/{id}/anular | @can('ventas.ver')<br>@can('ventas.ver')<br>@can('ventas.anular')<br>@can('ventas.anular') | ROBUSTO |
| **Cajas y Turnos**<br>`caja.blade.php` | Crear Caja<br>Aperturar Turno<br>Registrar Movimiento<br>Corte X<br>**Cierre Z (Ciego)** | POST /cajas<br>POST /cajas/turnos/aperturar<br>POST /cajas/movimientos<br>GET /cajas/turnos/{id}/reporte-x<br>POST /cajas/turnos/{id}/cerrar | @can('cajas.crear')<br>@can('cajas.aperturar')<br>@can('cajas.movimientos')<br>@can('cajas.arqueo')<br>@can('cajas.cerrar') | CONFORME |
| **Cuentas por Cobrar**<br>`cxc.blade.php` | Ver Estado Cuenta<br>Abonar a Factura<br>Abono Rápido FIFO<br>Ticket Abono | GET /cuentas-por-cobrar/cliente/{id}<br>POST /cuentas-por-cobrar/abonar-factura<br>POST /cuentas-por-cobrar/abonar-general<br>GET /cuentas-por-cobrar/ticket/{id} | @can('cxc.ver')<br>@can('cxc.abonar')<br>@can('cxc.abonar')<br>@can('cxc.ver') | OBS. JS |
| **Cuentas por Pagar**<br>`cxp.blade.php` | Ver Facturas Proveedor<br>Abonar Factura<br>Abono Rápido FIFO<br>Ticket Abono | GET /cuentas-por-pagar/proveedor/{id}<br>POST /cuentas-por-pagar/abonar-factura<br>POST /cuentas-por-pagar/abonar-general<br>GET /cuentas-por-pagar/ticket/{id} | @can('cxp.ver')<br>@can('cxp.abonar')<br>@can('cxp.abonar')<br>@can('cxp.ver') | OBS. JS |

| VISTA / MÓDULO | BOTONES EXISTENTES | RUTA HTTP ASOCIADA | DIRECTIVA @CAN / PERMISO | ESTADO |
|---|---|---|---|---|
| **Vendedores** `vendedor.blade.php` | Crear / Editar / Eliminar<br>Ver Comisiones Pendientes<br>**Liquidar Comisiones** | POST /vendedores<br>GET /vendedores/{id}/comisiones-pendientes<br>POST /vendedores/{id}/liquidar | @can('vendedores.crear')<br>@can('vendedores.ver')<br>@can('vendedores.liquidar') | CONFORME |
| **Kardex** `kardex.blade.php` | Ajuste de Stock (+/-)<br>Traslado entre Almacenes<br>Comprobante Kardex | POST /kardex/ajuste<br>POST /kardex/traslado<br>GET /kardex/comprobante/{id} | @can('inventario.ajustar_stock')<br>@can('inventario.traslados')<br>@can('inventario.kardex') | CONFORME |
| **Recepciones** `recepcion.blade.php` | Guardar Recepción<br>Guardar Borrador<br>**Anular Recepción** | POST /recepciones<br>POST /recepciones/borradores<br>POST /recepciones/{id}/anular | @can('compras.recepcion') | CONFORME |

**Detalle de la Observación en CXC y CXP:** Los modales de abono en Blade están correctamente protegidos con `@can('cxc.abonar')` y `@can('cxp.abonar')`. Sin embargo, las funciones que renderizan filas en `cxc.js` y `cxp.js` dibujan el botón de abono en la tabla sin evaluar permisos del cliente. Un usuario sin permiso de abono ve el botón y al presionarlo recibe error 403. Se recomienda condicionar la inyección del botón en JS.

## 5. HALLAZGOS CRÍTICOS, RIESGOS Y RECOMENDACIONES

### Hallazgo 1: Desconexión entre Cuentas por Pagar (CXP) y Flujo de Caja

**Diagnóstico:** En `CuentaPorPagarClass::abonarFacturaEspecifica()` y `abonarGeneralDeuda()`, los abonos a proveedores en efectivo no solicitan `caja_turno_id` ni generan un registro de tipo `egreso` en `caja_movimientos`.
**Riesgo:** Si se paga a un proveedor con dinero de la caja, el arqueo de cierre (Cierre Z) mostrará un sobrante contable irreal.
**Recomendación:** Exigir turno de caja cuando el abono a proveedor sea en efectivo e insertar automáticamente un movimiento de egreso en `caja_movimientos` con `documento_tipo = 'abono_cxp'`.

### Hallazgo 2: Detección Automática de Transacción en generarNumeroControl()

**Diagnóstico:** En `VentaClass.php`, `generarCodigo()` utiliza `$usarLock = $incrementar ?? (DB::transactionLevel() > 0)`, mientras que `generarNumeroControl()` tiene `$usarLock = $incrementar ?? false`.
**Riesgo:** Aunque en `procesarVenta()` se pasa explícitamente `true`, cualquier llamada externa futura dentro de una transacción que omita el parámetro generará lecturas sin bloqueo pesimista.
**Recomendación:** Unificar `generarNumeroControl()` para que adopte la misma lógica de `transactionLevel() > 0`.

### Hallazgo 3: Blindaje de Llaves Foráneas de Auditoría (Resuelto)

**Diagnóstico:** Se confirmó que las migraciones recientes aplicaron `restrictOnDelete` a las referencias de usuario en `ventas` y `caja_turnos`.

**Impacto:** Totalmente positivo. Ningún usuario con historial transaccional puede ser borrado de la base de datos, garantizando cumplimiento legal y auditorías históricas.

---

**FIN DEL INFORME DE AUDITORÍA TÉCNICA**

Generado por el Arquitecto de Software Senior y Auditor de Código